



Selection Task

Inter IIT Tech Meet ^{15.0}

Robotics Club

ROS 2 Humble

Gazebo Fortress

Nav2

OpenCV

Task : The Clue Chain Hunt

Team Size : Individual

Submission Deadline : Oct 12th, EOD



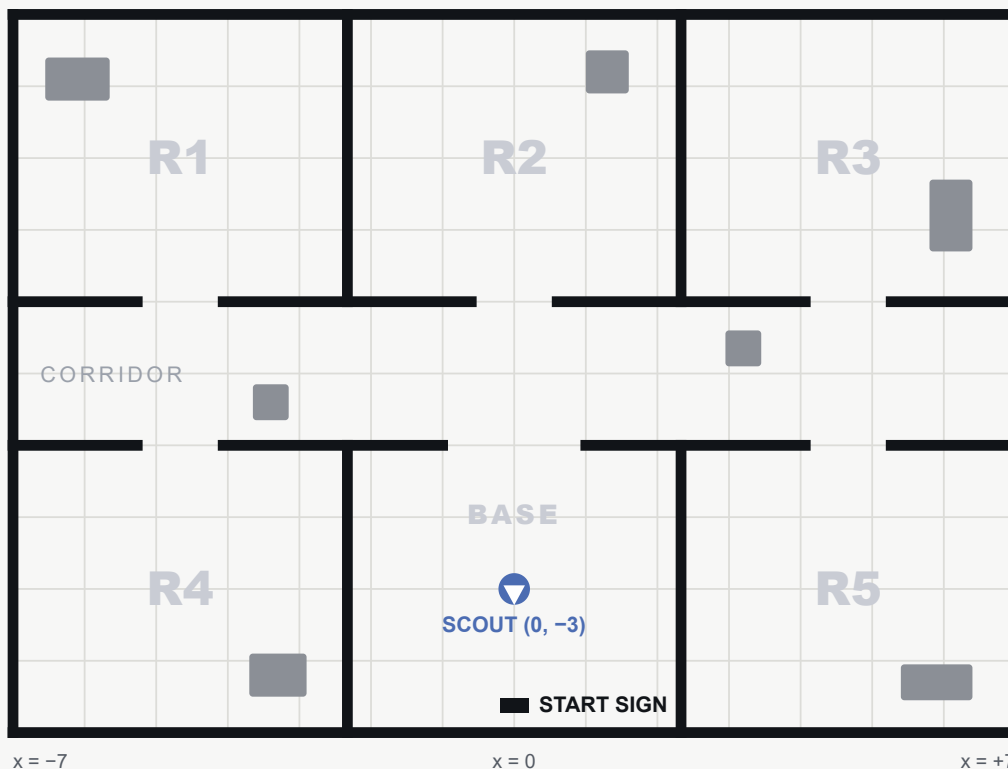


Problem Statement

Someone has hidden the Robotics Club's trophy somewhere in the lab building. All they left behind is a **trail of signs**, each carrying an ArUco marker. Every sign tells you the **colour of the room to search next**, but the rooms have no names on them. The only way to tell rooms apart is the **coloured pillar** standing inside each one.

Your robot, **Scout**, has to map the building, work out which room is which, follow the chain of clues sign by sign, and finish by parking precisely in front of the treasure marker. It must do all of this **on its own**, on a layout it has never seen before.

The Arena



14 m × 10 m, five rooms, a corridor and the base room. Walls and grey crates are the same in every run. Pillars, signs and the clue chain change with the **seed**.

- **Pillars:** one per room, colours red, green, blue, yellow and magenta. Radius 0.15 m, height 1.2 m.
- **Signs:** one per room plus the start sign. ArUco `DICT_4X4_50`, 0.30 m marker.
- **Scout:** diff drive, 360° LiDAR, RGB camera, IMU. It is provided in the starter repo.
- **Spawn:** (0, -3), facing the start sign.





Problem Statement

How the Hunt Works

THE CIPHER

A marker ID **0–39** is a clue. The next room's colour is `PALETTE[id % 5]`. A marker ID **40–49** means **the treasure is here**.

● 0 · red ● 1 · green ● 2 · blue ● 3 · yellow
● 4 · magenta

EXAMPLE

Start sign ID **19** → $19 \% 5 = 4$ → **magenta** room

Magenta room's sign ID **2** → **blue** room

Blue room's sign ID **31** → **green** room

Green room's sign ID **46** → **treasure!**

The chain visits 4 of the 5 rooms. The fifth room is a **decoy**: it has a sign with a valid-looking clue that is not part of the chain. Only follow the sign in the room you were sent to.

Objectives:

1. Mapping :

Launch the arena in mapping mode (`mapping==true` , walls and crates only). Build a 2D occupancy grid with `slam_toolbox` and save it. Every later step must reuse this map.

2. Localization & Navigation :

Bring up Nav2 with your saved map and AMCL. Tune the costmaps so Scout gets through the 1.2 m doorways and around the crates reliably. Handle goals that fail or get aborted.

3. Perception (OpenCV) :

Write a perception node that detects **(a)** the coloured pillars using HSV thresholding, morphology and contour filtering, and **(b)** ArUco markers and their IDs. Publish an annotated debug image on `/detections/image` and your detections on topics of your own design.





Problem Statement

Objectives (contd.):

4. Room Survey (TF) :

Before following any clue, visit the rooms and estimate each pillar's position in the `map` frame. Use the camera intrinsics for the bearing, the LiDAR or the pillar's known size for the range, and `tf2` for the transform. The result is a **colour** → **room** registry. Show the pillars as a `MarkerArray` in RViz2.

5. Follow the Clue Chain :

Read the start sign, decode it, drive to the room of that colour, **search the room for its sign**, read it, and repeat. Your node must decide on its own where to look and when a sign has been read reliably, e.g. by checking the same ID over several frames.

6. Treasure Docking :

When you read a treasure ID (40–49), park in front of that sign **0.6 m ± 0.1 m** from the board, facing it within $\pm 10^\circ$. Use **your own camera-based controller** (marker pose or centroid servoing). Nav2 must not be used for this final approach.

7. Hunt Log :

Write `hunt_log.yaml` containing the pillar positions, the sequence of marker IDs and rooms visited, the time of each step, and the final docking distance. Print a one-line summary, e.g. `19 → magenta → 2 → blue → 31 → green → 46 ★`

Optional (Bonus) :

- Identify and report the **decoy room** and its marker ID.
- Frontier-based exploration for the survey instead of hand-picked waypoints.
- A single launch file that runs Nav2, perception and the mission end-to-end.
- A table of your pillar estimates against the true positions (the organisers will provide these after submission).

Ground rules

Your stack will be run on a **secret seed** with an untouched copy of `clue_hunt_sim`. Use only the robot's sensors. Reading the generated world file, anything in `~/.clue_hunt/`, or Gazebo's ground-truth poses is disqualifying, as is hard-coding which colour is in which room. AI tools and online references are allowed, but you will be asked to explain any line of your submission in a short viva.





Problem Statement

Deliverables:

1. Technical Report (max 6 pages) :

- System overview, including an `rqt_graph` screenshot and the TF tree (`view_frames`).
- Your perception pipeline: HSV ranges, how you tuned them, and how you read markers robustly.
- The maths that turns a pixel plus a LiDAR range into a point in the `map` frame.
- Your mission state machine and docking controller, with a diagram.
- One real problem you hit, how you debugged it, and what fixed it.

2. Demo Video (max 5 min) :

Gazebo, RViz2 and the debug image visible together: the map, the survey with pillar markers appearing, the full clue chain, and the final dock, on **two different seeds**.

3. Code Package :

Your `clue_hunt_team` package: nodes, launch files, parameter YAMLS, the saved map, and a README with the exact run commands. It must build cleanly with `colcon build` .

Evaluation Criteria:

Aspect	Description	Weight
Mapping & Navigation	Map quality, AMCL + Nav2 tuning, reliable goals	20 %
Perception	Robust pillar + ArUco detection, debug output	20 %
Room Survey	Correct pixel → map transform, accuracy	15 %
Clue Chain Logic	Decoding, sign search, state machine, recovery	20 %
Treasure Docking	Visual servoing, final distance and heading	10 %
Secret-Seed Run	Completes the hunt on an unseen layout	5 %
Documentation & Viva	Report, README, ability to explain the code	10 %
	Total	100 %





Problem Statement

Additional Recommendations

Software Tools :

- ROS 2 Humble, Gazebo Fortress, RViz2
- OpenCV (`python3-opencv`), `rqt_image_view`

Packages :

- `slam_toolbox` , `nav2_bringup`
- `nav2_simple_commander` , `cv_bridge`
- `tf2_ros` , `visualization_msgs`

Suggested Steps:

1. Finish the warm-up exercises in the starter repo README.
2. Map once with `mapping:=true` , then get Nav2 reaching every room on several seeds.
3. Tune pillar and marker detection offline on recorded bags before running it live.
4. Get the pixel → map maths right for one pillar, then build the full registry.
5. Write the mission as a state machine: *SURVEY* → *READ* → *GOTO_ROOM* → *SEARCH_SIGN* → ... → *DOCK*.

Milestones:

Partial submissions are graded. Make each level solid before you move on.

LEVEL 1

Map & Move

Saved SLAM map, Nav2 + AMCL reaching every room, decoding the start sign.

LEVEL 2

See & Survey

Pillar + marker detection live, and a colour → room registry in the map frame.

LEVEL 3

Hunt & Dock

The full clue chain on any seed, visual-servo docking, and `hunt_log.yaml` .

Contact Details

Coordinator 1 : +91 XXXXX XXXXX

Coordinator 2 : +91 XXXXX XXXXX

